Physics-Informed Extreme Learning Machine Framework for Solving Linear Elasticity Mechanics Problems

Qimin Wang, Chao Li, Sheng Zhang, Chen Zhou, Yanping Zhou

1 Introduction

Linear elasticity mechanics plays a pivotal role in structural engineering, materials science, and mechanical design. Traditional numerical methods such as finite element method (FEM), finite difference method (FDM), and finite volume method (FVM) have inherent limitations in dealing with complex geometries, high computational cost, numerical stability, and accuracy. Recently, neural network-based methods, particularly Physics-Informed Neural Networks (PINNs), have been applied to computational mechanics. However, despite these advances, PINNs still present significant challenges in terms of prolonged training times, difficulties in selecting optimal network sizes, and limitations in solution accuracy and efficiency.

This paper proposes a novel framework that combines Physics-Informed Extreme Learning Machines (PIELM) with linear elastic mechanics, focusing on the integration of discretization operators with the principles of linear elasticity mechanics.

2 Theoretical Foundation

2.1 Basics of Linear Elasticity Mechanics

In linear elasticity problems, the solution is represented by a set of governing equations and boundary conditions. In the elastomer domain (Ω), the displacement field $\mathbf{u}$ satisfies the following PDEs (Equilibrium equation):

$$\mathbf{A}\boldsymbol{\sigma} + \mathbf{f} = 0 \tag{1}$$

where $\mathbf{A}$ is a differential operator, $\mathbf{f}$ is volume load vector, and $\boldsymbol{\sigma}$ is stress vector. Under small deformation, stress and strain conform to geometric equations:

$$\boldsymbol{\sigma} = D\boldsymbol{\epsilon} \tag{2}$$

where D is the elastics matrix, which is made of Lamé constants (G and λ), and $\boldsymbol{\epsilon}$ is the strain vector, which can be calculated by the geometric equation:

$$\boldsymbol{\epsilon} = L\mathbf{u} \tag{3}$$

where $L = \mathbf{A}^T$ is a differential operator.

In the elastic boundary ($\partial\Omega$), the governing equations must satisfy the following boundary conditions:

$$\begin{cases} \mathbf{u} = \bar{\mathbf{u}} \\ \mathbf{n}\boldsymbol{\sigma} = \mathbf{T} \end{cases} \tag{4}$$

where $\bar{\mathbf{u}}$ and $\mathbf{T}$ are determined by the displacement constraints and external load condition in the determined boundary, and $\mathbf{n}$ is the direction vector matrix of coordinate points.

2.2 Physics-Informed Extreme Learning Machine

The Extreme Learning Machine (ELM) is a feed-forward neural network learning algorithm proposed by Huang et al. (2006). Its core idea is to randomly initialize the weights and biases from the input layer to the hidden layer during the learning process, and then directly compute the weights from the hidden layer to the output layer by a parsing method.

The input layer consists of N input nodes, forming the input vector $\mathbf{x} = [x_1, x_2, \ldots, x_N]_{1\times N}$. The hidden layer contains L neurons, each employing an activation function $g(\cdot)$. The transformation from the input layer to the hidden layer is characterized by randomly generated weight matrix $\mathbf{w} = [w_1^T, w_2^T, \ldots, w_D^T]_{D\times L}^T$ and bias vector $\mathbf{b} = [b_1, b_2, \ldots, b_L]_{1\times L}$.

The output of each neuron h_i in the hidden layer is computed as:

$$h_i = g_i(\mathbf{x}w_i^T + b_i) \tag{5}$$

where g is the activation function. The output layer comprises M nodes, forming the output vector $\mathbf{u} = [u_1, u_2, \ldots, u_M]_{1\times M}$. The final output u_j for each output node is calculated by:

$$u_j = \mathbf{h}\beta_j \tag{6}$$

where $\mathbf{h} = [h_1, h_2, \ldots, h_L]_{1\times L}$ is the hidden neuron vector, and $\boldsymbol{\beta} = [\beta_1, \beta_2, \ldots, \beta_M]_{L\times M}$ is the weight matrix connecting the hidden neurons to the output nodes.

This entire process can be represented as:

$$\mathbf{u} = g(\mathbf{x}\mathbf{w} + \mathbf{b})\boldsymbol{\beta} \tag{7}$$

To enhance the capabilities of ELM in applications involving physical systems and scientific computing, the concept of Physics-Informed Extreme Learning Machines (PIELM) was introduced. PIELM integrates domain-specific physical laws and constraints into the ELM framework.

Considering a PDE:

$$\begin{cases} L(u(\mathbf{x})) = f(\mathbf{x}), & \mathbf{x} \in \Omega \\ B(u(\mathbf{x})) = h(\mathbf{x}), & \mathbf{x} \in \partial\Omega \end{cases} \tag{8}$$

where L is a differential operator and B is a boundary operator. In the context of PDEs, the higher derivatives of the output with respect to the input in L can be expressed as:

$$\frac{\partial^n u}{\partial x_i^n} = w_i^n \circ g^{(n)}(\mathbf{x}\mathbf{w} + \mathbf{b})\boldsymbol{\beta} \tag{9}$$

where $\circ$ is element-wise multiplication. For linear differential operators, it can be expressed as:

$$f(\mathbf{x}) = L(u(\mathbf{x})) = L(g(\mathbf{x}\mathbf{w} + \mathbf{b}))\boldsymbol{\beta}, \quad \mathbf{x} \in \Omega \tag{10}$$

The boundary equation can be expressed in ELM:

$$h(\mathbf{x}) = B(g(\mathbf{x}\mathbf{w} + \mathbf{b}))\boldsymbol{\beta}, \quad \mathbf{x} \in \partial\Omega \tag{11}$$

Thus, the residual term can be expressed as:

$$\xi = \left\| \begin{bmatrix} f(\mathbf{x}_i) \\ h(\mathbf{x}_j) \end{bmatrix} - \begin{bmatrix} L(g(\mathbf{x}_i\mathbf{w} + \mathbf{b})) \\ B(g(\mathbf{x}_j\mathbf{w} + \mathbf{b})) \end{bmatrix} \boldsymbol{\beta} \right\|_2^2, \quad \mathbf{x}_i \in \Omega, \mathbf{x}_j \in \partial\Omega \tag{12}$$

Let $K = [f(\mathbf{x}_i), h(\mathbf{x}_j)]^T$ and $H = [L(g(\mathbf{x}_i\mathbf{w} + \mathbf{b})), B(g(\mathbf{x}_j\mathbf{w} + \mathbf{b}))]^T$, $\boldsymbol{\beta}$ can be solved using the least squares method:

$$\boldsymbol{\beta} = \text{pinv}(H) \cdot K \tag{13}$$

where $\text{pinv}(H)$ is the generalized inverse of H.

3 PIELM Framework for Linear Elasticity Mechanics

3.1 Network Framework

In the PIELM used for solving linear elasticity mechanics problems, the framework is specifically designed to map spatial coordinates to displacement fields. The input layer takes spatial coordinates as inputs: (x, y, z) for a 3D problem and (x, y) for a 2D problem.

The output layer produces the displacement components corresponding to the input coordinates: (u, v, w) for a 3D problem and (u, v) for a 2D problem. The choice of activation function in the hidden layer is essential. In the context of linear elasticity problems, it is necessary to compute second-order derivatives. The tanh function has been chosen as the activation function because it exhibits a bounded range in its higher-order derivatives.

The PIELM framework for a 3D problem can be constructed as follows:

$$\text{Input layer:} \quad \mathbf{x} = [x, y, z] \in \mathbb{R}^3 \tag{14}$$

$$\text{Hidden layer:} \quad \mathbf{h} = \tanh(\mathbf{x}\mathbf{w} + \mathbf{b}) \in \mathbb{R}^L \tag{15}$$

$$\text{Output layer:} \quad \mathbf{u} = [u, v, w] = \mathbf{h}\boldsymbol{\beta} \in \mathbb{R}^3 \tag{16}$$

For a 2D problem, the input and output layers can be replaced with $\mathbf{x} = [x, y] \in \mathbb{R}^2$ and $\mathbf{u} = [u, v] = \mathbf{h}\boldsymbol{\beta} \in \mathbb{R}^2$.

3.2 Discretization Operators

The governing linear elasticity PDEs need to be discretized in the PIELM framework. This involves converting the continuous PDEs into a form that can be handled by the PIELM, with the equations then evaluated using collocation points.

The divergence of the stress tensor in terms of the displacement components can be expressed as:

$$\mathbf{A}DL\mathbf{u} + \mathbf{f} = 0 \tag{17}$$

in which, the elements of $\mathbf{A}DL$ can be represented as:

$$(ADL)_{ij} = (\lambda + G)\frac{\partial^2}{\partial x^{(i)} x^{(j)}} + \delta_{ij} G \sum_{k=1} \frac{\partial^2}{\partial x^{(k)2}} \tag{18}$$

where δ_{ij} is defined as:

$$\delta_{ij} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases} \tag{19}$$

and the 2-order partial derivative can be calculated by:

$$\frac{\partial^2 u^{(i)}}{\partial x^{(j)} \partial x^{(k)}} = w_j \circ w_k \circ \tanh''(\mathbf{x}\mathbf{w} + \mathbf{b})\beta_i \tag{20}$$

Therefore, the governing equation can be rewritten as:

$$\mathbf{K}\boldsymbol{\beta}' + \mathbf{f} = 0 \tag{21}$$

in which, $\boldsymbol{\beta}' = [\beta_1, \beta_2, \ldots, \beta_M]^T$, $\mathbf{K}$ is the governing discretization differential operator, and the element is obtained by replacing the 2-order partial derivative $\frac{\partial^2}{\partial x^{(i)} x^{(j)}}$ of $\mathbf{A}DL$ with $w_i \circ w_j \circ \tanh''(\mathbf{x}\mathbf{w} + \mathbf{b})$.

Similarly, the boundary conditions can be expressed in terms of displacements:

$$\begin{cases} \mathbf{P}\boldsymbol{\beta}' = \bar{\mathbf{u}} \\ \mathbf{Q}\boldsymbol{\beta}' = \mathbf{T} \end{cases} \tag{22}$$

where $\mathbf{P}$ is the displacement discretization operator, with elements:

$$P_{1i} = \mathbf{0} \text{ or } \mathbf{h} \tag{23}$$

in which, $\mathbf{h}$ is the vector of output from the hidden layer. $\mathbf{Q}$ is the stress discretization differential operator, with elements:

$$Q_{ij} = \lambda n_i w_j \circ \tanh'(\mathbf{x}\mathbf{w}+\mathbf{b}) + G n_j w_i \circ \tanh'(\mathbf{x}\mathbf{w}+\mathbf{b}) + \delta_{ij} G \sum_{k=1} n_k w_k \circ \tanh'(\mathbf{x}\mathbf{w}+\mathbf{b}) \tag{24}$$

3.3 Construction of the Loss Function

The loss function of PIELM consists of several parts, each corresponding to a different aspect of the problem. The primary constituent of the loss function is the residual of the governing PDEs, evaluated at the collocation points:

$$PDE = \sum_{x_i \in \Omega} \|\mathbf{K}_i \boldsymbol{\beta}' + \mathbf{f}_i\|^2 \tag{25}$$

where Ω denotes the set of collocation points in the intradomain, $\mathbf{K}_i$ represents the governing discretization differential operator applied at the collocation point $\mathbf{x}_i$, and $\mathbf{f}_i$ is the body force vector at x_i.

The loss function incorporates terms for both Dirichlet and Neumann boundary conditions. The residual for the Dirichlet boundary conditions is:

$$BC_u = \sum_{x_j \in \partial\Omega_u} \|\mathbf{P}_j \boldsymbol{\beta}' - \bar{\mathbf{u}}_j\|^2 \tag{26}$$

where $\partial\Omega_u$ denotes the set of boundary collocation points where displacements are prescribed, $\mathbf{P}_j$ is the displacement discretization operator at the collocation point $\mathbf{x}_j$, and $\bar{\mathbf{u}}_j$ is the prescribed displacement vector at $\mathbf{x}_j$.

Similarly, the residual for the Neumann boundary conditions is:

$$BC_T = \sum_{x_k \in \partial\Omega_T} \|\mathbf{Q}_k \boldsymbol{\beta}' - \mathbf{T}_k\|^2 \tag{27}$$

where $\partial\Omega_T$ denotes the set of boundary collocation points where tractions are prescribed, $\mathbf{Q}_k$ is the stress discretization differential operator at the collocation point $\mathbf{x}_k$, and $\mathbf{T}_k$ is the prescribed traction vector at $\mathbf{x}_k$.

The total loss function for the PIELM is a weighted sum of these components:

$$\mathcal{L} = PDE + \alpha_u BC_u + \alpha_T BC_T \tag{28}$$

where α_u and α_T are weights that balance the contributions of the boundary condition residuals relative to the PDE residuals. It can be assembled into a single linear system of equations:

$$\begin{bmatrix} \mathbf{K} \\ \alpha_u \mathbf{P} \\ \alpha_T \mathbf{Q} \end{bmatrix} \boldsymbol{\beta}' = \begin{bmatrix} -\mathbf{f} \\ \alpha_u \bar{\mathbf{u}} \\ \alpha_T \mathbf{T} \end{bmatrix} \tag{29}$$

This system can then be solved using the least squares method to find the optimal weights $\boldsymbol{\beta}'$.

3.4 Overall Workflow

The training process of the PIELM for solving linear elasticity mechanics problems comprises the following steps:

1. Collocation Points Generation - Intra-domain collocation points - Dirichlet boundary collocation points - Neumann boundary collocation points

2. Network Initialization - Randomly generate parameters weights $\mathbf{w}$ and bias $\mathbf{b}$ for the input layer

3. Loss Function Construction - Construct discretization operators to transform continuous problems into a computable discrete format - Use matrix representation to formalize the loss function

4. Calculate output weights - Solve for the output weights using the least squares method - Reshape output weights to fit output layers

5. Network Output - Integrate weights across input and output layers to compute the final network output - Calculate and present the displacement output, the result of network computations

4 Numerical Examples and Results

To demonstrate the efficacy of the proposed framework, three case studies are presented: two plate cases characterized by nonlinear loads and geometric defects, and a three-dimensional case with an analytical solution.

4.1 Case 1: Plate with Non-linear Distributed Load

The first case involves a 2×2 (m $\times$ m) square plate subjected to a distributed load along its vertical edges. The left and right sides are subjected to a nonlinear distributed load defined by $q = y^2$ (Pa), and the upper and lower sides are load-free. The Lamé constants of the material are $G = 30$ Pa and $\lambda = 20$ Pa. Due to the symmetry of the load and geometry, a quarter model is used for analysis.

Collocation points are distributed throughout the domain and boundaries. The PIELM and PINN methods are compared with the same number of collocation points and neurons, using FEM results as a reference solution. The results show that both PIELM and PINN closely replicate the FEM solution. However, PIELM tends to have smaller localized error zones, while PINN shows more dispersed error regions. The mean squared error (MSE) values of PIELM are lower than those of PINN for all displacement and stress components, indicating that PIELM performs better across the entire domain.

4.2 Case 2: Plate with Non-linear Geometric Defect

The second case involves a 2×2 (m $\times$ m) square plate with a circular defect of radius $r = 0.2$ m, subjected to a unified load along its vertical edges. The left and right sides are subjected to a unified distributed load defined

by $q = 1$ (Pa), and the upper and lower sides are load-free. The Lamé constants of the material are $G = 26.92$ Pa and $\lambda = 23.08$ Pa. Due to symmetry, a quarter model is used for analysis.

The PIELM and PINN methods are compared with the same setup as in Case 1. Both methods effectively capture the physical information of the concentrated stress around the defect. However, PIELM demonstrates superior overall accuracy compared to PINN, as evidenced by its lower MSE values for various displacement and stress components. The errors between the PIELM and PINN stress estimates are predominantly concentrated in the geometric defects, with PIELM showing a markedly increased error in the border region of the defect.

4.3 Case 3: Self-weight Elongation Problem with Analytical Solution

The third case addresses a classical problem of elongation under self-weight, using the analytical solution as a benchmark. The differential element in the entire cube is subjected to a gravitational force ρg, where ρ is the density and g is the gravitational acceleration. The parameters for analysis are: length of cube $l = 1$, density $\rho = 1$, elasticity modulus $E = 200$ Pa, and Poisson ratio $\mu = 0.2$.

The analytical solution for this problem is:

$$\begin{cases} u = -\frac{\mu\rho g x z}{E} \\ v = -\frac{\mu\rho g y z}{E} \\ w = \frac{\rho g(z^2+\mu(x^2+y^2)-l^2)}{2E} \end{cases} \tag{30}$$

The PIELM method is compared with FEM, and the results show that both methods can approximate the analytical solutions well. Multiple error norms, including L^2, H^1-semi, and energy norm, are employed to rigorously assess the accuracy of the solution. The comparison of error norms indicates that PIELM achieves lower values in both H^1-semi and energy norm compared to FEM, providing a more accurate representation of gradient and energy characteristics. In terms of the L^2 norm, both methods perform similarly.

5 Implications and Key Findings

5.1 Influence of Hidden Layer Neurons

The number of hidden layer neurons represents a crucial hyperparameter that directly impacts the expressiveness, convergence, and computational efficiency of the PIELM model. Experimental results show that a reduction in the number of neurons results in a notable increase in the MSE value, particularly for the stress component. As the number of neurons increases, the MSE declines and eventually stabilizes. This suggests that it is necessary to increase the number of hidden layer neurons to enhance the predictive accuracy of the model when dealing with complex structures or stress concentrations.

5.2 Stability of PIELM Solution with Random Initialization

Given that the weights in PIELM are generated through random initialization, multiple trials with input weights randomly generated from a standard normal distribution were conducted to evaluate stability. The results for both displacement and stress components demonstrate consistent performance with minimal fluctuations. These findings confirm that the performance of PIELM is not significantly dependent on the random initialization of parameters.

5.3 Effect of Local Refined Sampling

In regions exhibiting significant gradient variations, the distribution of collocation points directly influences the model's ability to discern the distinctive features of these regions. Experiments show that increasing the number of sampling points in regions with significant gradient variations gradually improves the model's accuracy in predicting stress. Once the number of sampling points reaches a certain threshold, the model's ability to capture complex local features significantly enhances, leading to a noticeable decrease in error.

5.4 Comparison Between PIELM and PINN

PIELM and PINN differ in several key aspects:

1. Architecture and Network Structure: PINN employs deep neural networks with multiple layers and complex structures, while PIELM uses a single-layer feed-forward neural network with a simple structure.

2. Hyperparameters: PINN has many hyperparameters that significantly impact model performance and are challenging to adjust. PIELM has fewer hyperparameters, primarily focusing on the number of hidden layer nodes and the activation function.

3. Gradient Calculation: PINN uses backpropagation for iterative optimization, which is complex and time-consuming. PIELM calculates gradients directly through the derivative formula of the activation function, making it faster and more efficient.

4. Training Process: PINN relies on iterative optimization and gradient descent, which is complex and time-consuming. PIELM directly solves the weights of the output layer using a single least squares problem, making it faster and more efficient.

5. Applicable Scenarios: PINN demonstrates reduced error in regions with significant gradient variations but has a relatively high overall error. PIELM demonstrates superior overall error control but has higher error rates in regions with significant gradient variations.

The PIELM framework offers several advantages for solving linear elasticity problems:

1. Computational Efficiency: PIELM avoids iterative optimization and directly solves for the output weights using the least squares method, resulting in significantly faster computation compared to PINN.

2. Accuracy: PIELM achieves high overall accuracy in predicting displacement and stress fields, with lower MSE values compared to PINN in the tested cases.

3. Stability: PIELM demonstrates stable performance across multiple random initializations of the input weights, indicating robustness in its predictions.

4. Adaptability: PIELM can be adapted to various boundary conditions and loading scenarios, as demonstrated in the case studies.

5. Superiority over FEM in certain aspects: PIELM achieves lower H^1-semi and energy norm errors compared to FEM in the three-dimensional case, indicating better accuracy in gradient and energy characteristics.

These findings suggest that the PIELM framework provides an efficient and accurate alternative for solving linear elasticity problems, particularly when dealing with complex geometries and boundary conditions.